When a user is authenticated, the user will have a set of claims that are used to store the information about the user. For example, the user can have a claim that specifies the user's role. So, technically, roles are also claims, but they are special claims that are used to store the roles of the user. We can store other information in the claims, such as the user's name, email address, date of birth, driving license number, and more. Once we've done this, the authorization system can check the claims to determine whether the user is allowed to access the resource. Claim-based authorization provides more granular access control than role-based authorization, but it can be more complex to implement and manage.

****

A claim is a key-value pair. Note that the claim type and value are case-sensitive. You can store the claims in the database or the code. When the user logs in, the claims can be retrieved from the database and added to the token.

****

```
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("RequireAdministratorRole", policy =>
policy.RequireRole(AppRoles.Administrator));
    options.AddPolicy("RequireVipUserRole", policy =>
policy.RequireRole(AppRoles.VipUser));
    options.AddPolicy("RequireUserRole", policy =>
policy.RequireRole(AppRoles.User));
    options.AddPolicy("RequireUserRoleOrVipUserRole", policy =>
policy.RequireRole(AppRoles.User, AppRoles.VipUser));
    options.AddPolicy(AppAuthorizationPolicies.RequireAccessNumber, policy =>
policy.RequireClaim(AppClaimTypes.AccessNumber));
    options.AddPolicy(AppAuthorizationPolicies.RequireDrivingLicenseNumber, policy
=> policy.RequireClaim(AppClaimTypes.DrivingLicenseNumber));
    // Here we can set multiple values not Russia
    options.AddPolicy(AppAuthorizationPolicies.RequireCountry, policy =>
policy.RequireClaim(ClaimTypes.Country, "Russia", "Syria"));
});
```

****

The AppClaimTypes:

```
namespace JLokaAuthentication.Authentication
{
    public class AppClaimTypes
    {
        public const string DrivingLicenseNumber = "DrivingLicenseNumber";
        public const string AccessNumber = "AccessNumber";
    }
}
```

****

The Authorization Policies:

```
namespace JLokaAuthentication.Authentication
{
    public static class AppAuthorizationPolicies
    {
        public const string RequireDrivingLicenseNumber =
"RequireDrivingLicenseNumber";
        public const string RequireAccessNumber = "RequireAccessNumber";
        public const string RequireCountry = "RequireCountry";
    }
}
```

****

To create conditional policy we can use:

```
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("RequireAdministratorRole", policy =>
policy.RequireRole(AppRoles.Administrator));
    options.AddPolicy("RequireVipUserRole", policy =>
policy.RequireRole(AppRoles.VipUser));
    options.AddPolicy("RequireUserRole", policy =>
policy.RequireRole(AppRoles.User));
    options.AddPolicy("RequireUserRoleOrVipUserRole", policy =>
policy.RequireRole(AppRoles.User, AppRoles.VipUser));
    options.AddPolicy(AppAuthorizationPolicies.RequireAccessNumber, policy =>
policy.RequireClaim(AppClaimTypes.AccessNumber));
    options.AddPolicy(AppAuthorizationPolicies.RequireDrivingLicenseNumber, policy
=> policy.RequireClaim(AppClaimTypes.DrivingLicenseNumber));
    // Here we can set multiple values not Russia
    options.AddPolicy(AppAuthorizationPolicies.RequireCountry, policy =>
policy.RequireClaim(ClaimTypes.Country, "Russia", "Syria"));
    options.AddPolicy(AppAuthorizationPolicies.RequireDrivingLicenseAndAccessNumber,
policy => policy.RequireAssertion(context =>
    {
        var hasDrivingLicense = context.User.HasClaim(c => c.Type ==
AppClaimTypes.DrivingLicenseNumber);
        var hasAccessNumber = context.User.HasClaim(c => c.Type ==
AppClaimTypes.AccessNumber);

        return hasDrivingLicense && hasAccessNumber;
    }));
});
```

****

```
public void AddPolicy(string name, Action<AuthorizationPolicyBuilder>
configurePolicy)
{
    // Omitted for brevity
}
```

The `AddPolicy()` method accepts two parameters:

- A `name` parameter, which is the name of the policy
- A `configurePolicy` parameter, which is a delegate that accepts an `AuthorizationPolicyBuilder` parameter

****

```
[Authorize(Policy = AppAuthorizationPolicies.SpecialPremiumContent)]
[HttpGet("get-premium")]
public IActionResult GetPremium()
{
    return NoContent();
}
```

```
using Microsoft.AspNetCore.Authorization;

namespace JLokaAuthentication.Authentication
{
    public class SpecialPremiumContentRequirement : IAuthorizationRequirement
    {
        public string Country { get; set; } = string.Empty;

        public SpecialPremiumContentRequirement(string country)
        {
            Country = country;
        }
    }
}
```

```
using Microsoft.AspNetCore.Authorization;
using System.Security.Claims;

namespace JLokaAuthentication.Authentication
{
    public class SpecialPremiumContentAuthorizationHandler:
AuthorizationHandler<SpecialPremiumContentRequirement>
    {
        protected override Task HandleRequirementAsync(AuthorizationHandlerContext
context, SpecialPremiumContentRequirement requirement)
        {
            Console.WriteLine("Executed...");
            var hasPremiumAccess = context.User.HasClaim(c => c.Type ==
AppClaimTypes.Subscription && c.Value == "Premium");
            if(!hasPremiumAccess)
            {
                return Task.CompletedTask;
            }
            var countryClaim = context.User.FindFirst(c => c.Type ==
ClaimTypes.Country);
            if (countryClaim == null ||
String.IsNullOrEmpty(countryClaim.ToString()))
            {
                return Task.CompletedTask;
            }

            if(countryClaim.Value == requirement.Country)
            {
                context.Succeed(requirement);
            }
            return Task.CompletedTask;
        }
    }
```

```
}
```

And in the app builder:

```
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("RequireAdministratorRole", policy =>
policy.RequireRole(AppRoles.Administrator));
    options.AddPolicy("RequireVipUserRole", policy =>
policy.RequireRole(AppRoles.VipUser));
    options.AddPolicy("RequireUserRole", policy =>
policy.RequireRole(AppRoles.User));
    options.AddPolicy("RequireUserRoleOrVipUserRole", policy =>
policy.RequireRole(AppRoles.User, AppRoles.VipUser));
    options.AddPolicy(AppAuthorizationPolicies.RequireAccessNumber, policy =>
policy.RequireClaim(AppClaimTypes.AccessNumber));
    options.AddPolicy(AppAuthorizationPolicies.RequireDrivingLicenseNumber, policy
=> policy.RequireClaim(AppClaimTypes.DrivingLicenseNumber));
    // Here we can set multiple values not Russia
    options.AddPolicy(AppAuthorizationPolicies.RequireCountry, policy =>
policy.RequireClaim(ClaimTypes.Country, "Russia", "Syria"));
    options.AddPolicy(AppAuthorizationPolicies.RequireDrivingLicenseAndAccessNumber,
policy => policy.RequireAssertion(context =>
    {
        var hasDrivingLicense = context.User.HasClaim(c => c.Type ==
AppClaimTypes.DrivingLicenseNumber);
        var hasAccessNumber = context.User.HasClaim(c => c.Type ==
AppClaimTypes.AccessNumber);

        return hasDrivingLicense && hasAccessNumber;
    }));
    // Adding custom policy implementation
    options.AddPolicy(AppAuthorizationPolicies.SpecialPremiumContent, policy =>
    {
        policy.Requirements.Add(new SpecialPremiumContentRequirement("Russia"));
    });
});

builder.Services.AddSingleton<IAuthorizationHandler,
SpecialPremiumContentAuthorizationHandler>();
```

****

There are some points to note for policy-based authorization:

- You can use one `AuthorizationHandler` instance to handle multiple requirements. in the `HandleAsync()` method, you can use `AuthorizationHandlerContext.PendingRequirements` to get all the pending requirements and then check them one by one.
- If you have multiple `AuthorizationHandler` instances, they will be invoked in any order, which means you cannot expect the order of the handlers.
- You need to call `context.Succeed(requirement)` to mark the requirement as satisfied.

****

What if the requirement is not satisfied? There are two options:

- Generally, you do not need to call `context.Fail()` to mark the failed requirement because there may be other handlers to handle the same requirement, which may be satisfied.
- If you want to make sure the requirement fails and indicate that the whole authorization process fails, you can call `context.Fail()` explicitly, and set the `InvokeHandlersAfterFailure` property to `false` in the `AddAuthorization()` method, like this:

```
builder.Services.AddAuthorization(options =>
  {
      options.AddPolicy(AppAuthorizationPolicies.PremiumContent,
policy =>
      {
          policy.Requirements.Add(new
PremiumContentRequirement("New Zealand"));
      });
      options.InvokeHandlersAfterFailure = false;
  });
```

****

Method	**Description**
`CreateAsync(TUser user, string password)`	Creates a user with the given password.
`UpdateUserAsync(TUser user)`	Updates a user.
`FindByNameAsync(string userName)`	Finds a user by name.
`FindByIdAsync(string userId)`	Finds a user by ID.
`FindByEmailAsync(string email)`	Finds a user by email.
`DeleteAsync(TUser user)`	Deletes a user.
`AddToRoleAsync(TUser user, string role)`	Adds the user to a role.
`GetRolesAsync(TUser user)`	Gets a list of roles for the user.

`IsInRoleAsync(TUser user, string role)`	Checks whether the user has a role.
`RemoveFromRoleAsync(TUser user, string role)`	Removes the user from a role.
`CheckPasswordAsync(TUser user, string password)`	Checks whether the password is correct for the user.
`ChangePasswordAsync(TUser user, string currentPassword, string newPassword)`	Changes the user's password. The user must provide the correct current password.
`GeneratePasswordResetTokenAsync(TUser user)`	Generates a token for resetting the user's password. You need to specify `options.Token.PasswordResetTokenProvider` in the `AddIdentityCore()` method.

Method	Description
`GenerateEmailConfirmationTokenAsync(TUser user)`	Generates a token for confirming the user's email. You need to specify `options.Tokens.EmailConfirmationTokenProvider` in the `AddIdentityCore()` method.
`ConfirmEmailAsync(TUser user, string token)`	Checks whether the user has a valid email confirmation token. If the token matches, this method will set the `EmailConfirmed` property of the user to `true`.

****

Here are some common operations for managing roles by using the `RoleManager` class:

Method	Description
`CreateAsync(TRole role)`	Creates a role
`RoleExistsAsync(string roleName)`	Checks whether the role exists
`UpdateAsync(TRole role)`	Updates a role
`DeleteAsync(TRole role)`	Deletes a role
`FindByNameAsync(string roleName)`	Finds a role by name

Table 8.2 – Common operations for managing roles

****

These APIs encapsulate the database operations, so we can use them to manage users and roles easily. Some of the methods return a Task object. The IdentityResult object contains a Succeeded property to indicate whether the operation is successful. If the operation is not successful, you can get the error messages by using the Errors property.

****

```
// Add the identity model and attach to db context
builder.Services.AddIdentityCore<AppUser>()
    // this is not added by default
    .AddSignInManager() //
    //  If you use the AddIdentity() method, you do not need to call the AddRoles()
method.
    // The AddIdentity() method will call the AddRoles() method internally.
    .AddRoles<IdentityRole>()
    .AddEntityFrameworkStores<AppDbContext>()
    .AddDefaultTokenProviders();
```

```
[HttpPost("login")]
public async Task<IActionResult> Login([FromBody] LoginModel model)
{
    // Get the secret in the configuration
    // Check if the model is valid
    if(ModelState.IsValid)
    {
        var user = await userManager.FindByNameAsync(model.Username);
        if(user != null)
        {
            //if (await userManager.CheckPasswordAsync(user, model.password))
            var result = await signInManager.CheckPasswordSignInAsync(user,
model.password, lockoutOnFailure: true);
            if (result.Succeeded)
            {
                var token = GenerateToken(model.Username, user);
                return Ok(new { token });
            }
        }
        // If the user is not found, display an error message
        ModelState.AddModelError("", "Invalid username or password");
    }
    return BadRequest(ModelState);
}
```

****

ASP.NET Core provides a built-in model binding and model validation mechanism. Model binding is used to convert the data sent by the client to the corresponding model. The data sent by the client can be in different formats, such as JSON, XML, form fields, or query strings. Model validation is used to check whether the data sent by the client is valid. We used model validation in the previous sections. For example, here is the code we used to register a new user:

```
// Create an action to register a new user
[HttpPost("register")]
public async Task<IActionResult> Register([FromBody]
AddOrUpdateAppUserModel model)
{
    // Check if the model is valid
    if (ModelState.IsValid)
    {
        // Omitted for brevity
    }
    return BadRequest(ModelState);
}
```

****

The `ModelState.IsValid` property represents whether the model is valid. So, how does ASP.NET Core validate the model? Look at the `AddOrUpdateAppUserModel` class:

```
public class AddOrUpdateAppUserModel
{
    [Required(ErrorMessage = "User name is required")]
    public string UserName { get; set; } = string.Empty;

    [EmailAddress]
    [Required(ErrorMessage = "Email is required")]
    public string Email { get; set; } = string.Empty;

    [Required(ErrorMessage = "Password is required")]
    public string Password { get; set; } = string.Empty;
}
```

We use the validation attributes to specify the validation rules. For example, `Required` is a built-in attribute annotation that specifies that the property is required. Here are some of the most commonly used ones besides `Required`:

****

We use the validation attributes to specify the validation rules. For example, `Required` is a built-in attribute annotation that specifies that the property is required. Here are some of the most commonly used ones besides `Required`:

- `CreditCard`: This specifies that the property must be a valid credit card number
- `EmailAddress`: This specifies that the property must be a valid email address
- `Phone`: This specifies that the property must be a valid phone number
- `Range`: This specifies that the property must be within a specified range
- `RegularExpression`: This specifies that the property must match a specified regular expression
- `StringLength`: This specifies that the property must be a string with a specified length
- `Url`: This specifies that the property must be a valid URL
- `Compare`: This specifies that the property must be the same as another property